Repository Notes: EGMN (Watch Time Prediction)

1 What is in this repository?

This repository is a PyTorch research codebase for **video watch-time prediction**. It contains an implementation of the paper:

> *"Multi-Granularity Distribution Modeling for Video Watch Time Prediction via Exponential-Gaussian Mixture Network"*

The repository also includes several baseline models and code to preprocess the public KuaiRec dataset.

1.1 High-level structure

- `model/`: Model implementations (proposed EGMN and baselines).
- `dataloader/`: Dataset loaders that read preprocessed files and provide PyTorch `DataLoader`s.
- `dataset/kuairec/`: KuaiRec preprocessing scripts.
- `run_*.py`: Entrypoints to train/evaluate each method.
- `utils.py`: Evaluation metrics (MAE, XAUC, KL divergence) and helper utilities.

1.2 Evaluation outputs

The run scripts train on `train` and evaluate on `test`, printing (at least):

- **MAE**: mean absolute error on watch-time prediction.
- **XAUC**: a ranking-style metric computed from label–prediction ordering.
- **KL**: KL divergence between binned empirical distributions.

2 What is the model (EGMN)?

The main proposed model in this repository is **EGMN** (Exponential–Gaussian Mixture Network), implemented in `model/egmn.py`. EGMN models watch time as a **mixture distribution** designed to capture both short "quick-skip" behavior and longer watch-time behavior.

2.1 Distribution family

EGMN uses a mixture with:

- **One Exponential component** (to capture the spike near zero watch time).
- **Multiple Gaussian components** (to capture longer watch time). In the code, the Gaussian components are implemented as **Normals truncated at 0** (to avoid negative time).

If we denote the mixture weights by π, exponential rate by λ, and Gaussian parameters (μ_k, σ_k), the model defines a mixture density over watch time $y \geq 0$ of the form:

$$p(y) = \pi_0 \operatorname{Exp}(y; \lambda) + \sum_{k=1}^{K} \pi_k \operatorname{TN}(y; \mu_k, \sigma_k, \text{lower} = 0),$$

where $\operatorname{TN}(\cdot)$ is a Normal distribution truncated below at 0.

2.2 Neural architecture

- **Inputs**: a feature dictionary produced by the dataloader (sparse IDs, sequence features with masks, and continuous features such as `duration`).
- **Feature encoding**: embeddings for sparse/sequence features, and linear transforms for continuous features.
- **Shared trunk**: a multi-layer perceptron (MLP) over the concatenated feature representation.
- **Output heads**: predict mixture logits (converted to weights by softmax), the exponential rate λ (via softplus), and Gaussian parameters μ, σ (also via softplus for positivity constraints in the implementation).

2.3 Training objective (as implemented)

Training uses a combination of:

- **Negative log-likelihood (NLL)** of the observed watch time under the mixture.
- **Reconstruction regularizer**: an L1 loss between the observed label y and the model's mixture *mean* prediction.
- **Mixture entropy term**: computed from the mixture weights π, weighted by a hyperparameter (`-alpha` in `run_egmn.py`).

In the training script, the total loss is:

$$\mathcal{L} = \mathcal{L}_{\text{NLL}} + \alpha \, \mathcal{L}_{\text{entropy}} + \beta \, \mathcal{L}_{\text{reg}},$$

with α and β controlled by command-line flags.

2.4 Prediction used for evaluation

For evaluation, the code uses a **point prediction equal to the mixture mean**. In the implementation, the predicted value is computed as a weighted sum of component means (including the exponential mean $1/\lambda$).

3 Workflow: dataset preprocessing and running experiments

3.1 Dataset preprocessing (KuaiRec)

The KuaiRec raw data must be preprocessed before training. The provided script in `dataset/kuairec/kuairec_pr`

- merges multiple KuaiRec feature CSVs into a single table,
- builds vocabularies for sparse/sequence features,
- constructs a feature `description` schema used by models and dataloaders,
- splits into `train`/`test`,
- writes `kuairec_data.pkl` containing the processed data and schema.

  It also produces duration buckets and per-bucket play-time quantiles used by the D2Q baseline.

3.2 Training / evaluation entrypoints

Each method has a `run_*.py` entrypoint. Typical usage is:

- `python run_egmn.py -dataset_name kuairec`
- `python run_cread.py -dataset_name kuairec`
- `python run_d2q.py -dataset_name kuairec`

  These scripts:

- load `./dataset/kuairec/kuairec_data.pkl` through `dataloader/kuairec.py`,
- train for a fixed number of epochs,
- evaluate on the test set,
- print MAE, XAUC, and KL divergence.

3.3 Dependencies

The README indicates the project was tested with Python 3.x and PyTorch (e.g. 2.1.0), and expects common scientific Python packages (NumPy, pandas, scikit-learn).